GPT-6 ASTRA COURSE CHEATSHEET

O'Reilly Live Training — Getting Started with GPT-6 Astra

Instructor: Lucas | Responses API · Parameters · Tools · Prompting · Agents | 2026

1. MODEL VARIANTS (ASTRA / 6.1 SOL / LUNA, RELEASED 2026-09-03 → 09-29)

gpt-6-astra FLAGSHIP

- $10.00 / $50.00 per 1M tokens (in/out); cached input $1.00
- Hardest reasoning, coding, professional & agentic work
- Effort `low` → `max` — no `none`
- Replace: `gpt-5.6-sol`

gpt-6.1-sol

- $2.00 / $10.00 per 1M tokens (in/out); cached input $0.10
- Near-Astra performance at lower cost
- Replaced GPT-6 Sol as the mid tier on 2026-09-29
- Replace: `gpt-5.6-terra`

gpt-6-luna

- $0.10 / $0.50 per 1M tokens (in/out); cached input $0.01
- Fast, high-volume, focused tasks
- Only GPT-6 tier that accepts effort `none`
- Replace: `gpt-5.6-luna`

⚠ Context / Pricing Notes (all three tiers)

Context window **1,050,000** tokens (922K max input on Astra / 6.1 Sol), max output **128,000** tokens. Batch and Flex run at 50% of standard; Fast mode is ~2.5x speed at 2x price. Knowledge cutoff 2026-04-30 (Astra, 6.1 Sol). Endpoints: Responses, Chat Completions, Batch — not Realtime, Assistants, Embeddings, or fine-tuning. Mapping table: **developers.openai.com/api/docs/guides/latest-model**.

2. RESPONSES API (RECOMMENDED)

Basic Call

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-6-astra",
    input="Your prompt here",
    reasoning={"effort": "medium"},
    text={"verbosity": "medium"}
)
print(response.output_text)
```

Multi-turn via Conversations API (current pattern)

```
conversation = client.conversations.create()

first = client.responses.create(
    model="gpt-6-astra",
    conversation=conversation.id,
    input="Design an LRU cache in Python",
    reasoning={"effort": "high"}
)

# Follow-up — shares context via conversation id
follow_up = client.responses.create(
    model="gpt-6-astra",
    conversation=conversation.id,
    input="Add thread-safety."
)
# Legacy chaining (still works): previous_response_id=first.id
```

Why Responses API over Chat Completions?

- **CoT persistence** via `conversation=` id (or legacy `previous_response_id`)
- Fewer reasoning tokens re-generated
- Higher cache hit rates
- Lower latency on follow-ups
- Conversation-attached responses persist indefinitely (no 30-day TTL)

❌ Rejected with Reasoning (GPT-6)

- `temperature` — rejected when effort ≠ `none`
- `top_p` — rejected when effort ≠ `none`
- `top_logprobs` (and `logprobs` on Chat Completions) — rejected

Use instead:

- `reasoning: {effort: ...}`
- `text: {verbosity: ...}`
- `max_output_tokens`

3. REASONING EFFORT PARAMETER

reasoning={"effort": "..."} — Default: "medium" (Astra / 6.1 Sol)

Value	Speed	Reasoning Tokens	Best For
`"none"`	Fastest TTFT	None	**Luna only** — latency-critical retrieval, simple classification
`"low"`	Fast	Light	Lowest on Astra / 6.1 Sol — quick responses
`"medium"`	Balanced	Moderate	Default — general tasks
`"high"`	Slow	Most	Complex math, multi-step planning, difficult code
`"xhigh"`	Slower	Very high	Hardest async/agentic tasks (top tier added in GPT-5.5)
`"max"`	Slowest	Highest	Hardest problems

💡 **New GPT-6 API Controls (launched with Astra, 2026-09-03)**

Async tool calling — mark a function or custom tool with `async: true`; the model keeps working and you return the result when it is ready. **Mid-turn steering** — send extra user instructions while the model is still working (WebSocket mode). **Change reasoning effort mid-conversation** — a `configuration_update` input item adjusts settings without losing the prompt cache. Also listed: Programmatic Tool Calling, multi-agent orchestration, persisted reasoning, compaction, pro mode, Fast / Ultrafast modes. Check the per-feature guides for exact request syntax.

4. VERBOSITY PARAMETER

text={"verbosity": "..."} — Default: "medium"

Value	Output Length	Best For
`"low"`	Lowest of the three	Terse UX, SQL queries, concise code, minimal prose
`"medium"`	Baseline	Balanced explanations (default)
`"high"`	Highest of the three	Audits, teaching, hand-offs, detailed documentation

💡 **Measure, Don't Assume — and Mixed Verbosity Strategy**

Exact token counts per verbosity level vary by prompt and model generation — benchmark live against your own prompts rather than relying on fixed numbers. The trend (low < medium < high) is stable across the GPT-5.x line. Cursor-style pattern: set global `verbosity="low"` for prose, but instruct "use high verbosity for code tools" in the system prompt to get concise explanations + detailed, well-commented code.

5. NEW TOOL FEATURES

Freeform / Custom Tool Calling (NEW)

```
# Raw text payloads — no JSON wrapping
response = client.responses.create(
    model="gpt-6-astra",
    input="Use code_exec to find
factorial of 10.",
    tools=[{
        "type": "custom",
        "name": "code_exec",
        "description": "Executes
arbitrary Python code"
    }]
)
# Access: response.output[1].input
(raw code string)
# Feed back: {"type":
"function_call_output",
#              "call_id":
tool_call.call_id,
#              "output":
"result_string"}
```

Context-Free Grammar (CFG) Constraints (NEW)

```
# Constrain output to exact syntax
(Lark or Regex)
tools=[{
    "type": "custom",
    "name": "sql_tool",
    "description": "SQL tool — reason
to conform",
    "format": {
        "type": "grammar",
        "syntax": "lark",  # or "regex"
        "definition": r"""
start: "SELECT" SP cols SP "FROM"
       SP table SP "WHERE" SP filter
cols: IDENTIFIER ("," SP IDENTIFIER)*
IDENTIFIER: /[A-Za-z_][A-Za-z0-9_]*/
SP: " "
"""
    }
}]
```

Freeform Tool — Key Points

- Use for Python, SQL, shell, config — anything where JSON wrapping is unnatural
- Model emits `ResponseCustomToolCall` with `.input` = raw string
- ⚠ Does **NOT** support parallel tool calling
- Great for: code sandboxes, SQL databases, shell envs
- Feed result back via `function_call_output` to continue

CFG — Key Points & Best Practices

- Lark: terminals (UPPER) matched first; rules (lower) combine them
- Regex: provide pattern string directly
- ✅ One bounded terminal for free-text between anchors
- ❌ Don't split free-text across multiple terminals
- Use `SP = " "` explicitly; avoid `.*` wildcards
- If drifting OOD → tighten grammar + bump reasoning effort

Allowed Tools — Selective Access (NEW)

- Define full toolkit, restrict active subset per request
- Use for safety, predictability, preventing unintended calls

```
tool_choice={"type":"allowed_tools","mode":"auto","tools":[{"type":"function","name":"safe_tool"}]}
```

Tool Preambles

- GPT-5 trained to give upfront plans + progress updates
- Trigger: "Before calling a tool, explain why"
- System prompt: "Rephrase goal, outline plan, narrate each step, summarize at end"
- Useful for debugging and transparency

6. MIGRATION GUIDE

Migrating From GPT-5.6 to GPT-6

GPT-5.6	GPT-6	Notes
`gpt-5.6-sol`	`gpt-6-astra`	`reasoning.effort: "none"` → `"low"` (only Luna accepts `"none"`)
`gpt-5.6-terra`	`gpt-6.1-sol`	Same effort levels as Astra: `low` → `max`, default `medium`
`gpt-5.6-luna`	`gpt-6-luna`	`"none"` still supported
`prompt_cache_retention`	`prompt_cache_options.ttl`	Parameter renamed

7. PROMPTING BEST PRACTICES

Controlling Agentic Eagerness

Less eager (quick, focused):

- Start broad, fan out to focused subqueries in parallel
- Stop as soon as you can name exact content to change
- Early stop: top hits converge ~70% on one area
- Batch search → minimal plan → complete task

More eager (autonomous, persistent):

- "Keep going until query is completely resolved"
- "Never stop when encountering uncertainty"
- "Decide, proceed, document assumptions for user"

Instruction Following Rules

- Keep instructions **non-contradictory**
- Priority order: Security (CRITICAL) → Logic (HIGH) → Performance (MEDIUM) → Style (LOW)
- Specify output format explicitly
- Use structured system prompts with clear sections
- Markdown not used by default — must prompt for it
- Inline math: `\( \)`, block math: `\[ \]`

Self-Reflection for High Quality

Add to system prompt:

- Create a quality rubric with 5-7 categories internally
- Evaluate and iterate until hitting top marks in ALL categories
- Don't show rubric to user — it's for internal reasoning
- Great for: one-shot app generation, frontend code, complex outputs

Metaprompting

Ask GPT-5 to improve its own prompts:

- *"Here's a prompt: {prompt}. Desired behavior: {X}. It instead {Y}. What minimal edits encourage improvement?"*
- Use `reasoning.effort: "high"` for metaprompting tasks
- Keep as much of the original prompt intact as possible
- Ask for specific phrases to add or remove

8. AGENTIC CAPABILITIES

Web Search Tool

```
response = client.responses.create(
    model="gpt-6-astra",
    tools=[{"type": "web_search"}],
    input="Latest AI tool releases this quarter?"
)
print(response.output_text)

# Chain: Markdown table from results (cost-optimized follow-up)
table_response = client.responses.create(
    model="gpt-6.1-sol",
    previous_response_id=response.id,
    input=[{"role": "user",
            "content": "Put results in a markdown table."}]
)
```

Multi-turn Agentic Loop

```
def run_conversation(messages, prev_id=None):
    response = client.responses.create(
        model="gpt-6-astra",
        input=messages,
        tools=TOOLS,
        previous_response_id=prev_id
    )
    tool_call = response.output[1] \
        if len(response.output) > 1 else None

    if tool_call and tool_call.type=="custom_tool_call":
        result = execute_tool(tool_call.input)
        messages.append({
            "type": "function_call_output",
            "call_id": tool_call.call_id,
            "output": result
        })
        return run_conversation(messages, response.id)
    return response  # base case: no more tool calls
```

Retail Agent Pattern

1. Authenticate user (email or name+zip)
2. Retrieve relevant data
3. Confirm before consequential actions
4. Execute with `reasoning: "low"` (`"none"` on Luna)
5. One tool call at a time

SWE-Bench Pattern

- Use `reasoning: "high"`
- Verify changes extremely thoroughly
- Account for hidden test edge cases
- Make as many tool calls as needed
- Triple-check before ending turn

Frontend Dev Pattern

- Stack: Next.js + TypeScript + Tailwind + shadcn/ui
- Icons: Lucide, Animation: Motion
- 4px spacing multiples, max 4-5 font sizes
- Skeletons for loading states
- Self-reflection rubric for quality

9. QUICK REFERENCE & GOTCHAS

✅ DO THIS

- Use **Responses API** (`client.responses.create`), not Chat Completions
- Use `client.conversations.create()` for multi-turn (legacy `previous_response_id` still works)
- Use `gpt-6-luna` with `reasoning.effort: "none"` for speed-critical paths
- Use `text.verbosity: "high"` for thorough code docs/audits
- Keep CFG grammars **simple** — complex ones get API-rejected
- Set `parallel_tool_calls=False` with custom tools
- Use `gpt-6.1-sol` or `gpt-6-luna` for cost-sensitive follow-up processing of Astra outputs

❌ AVOID THIS

- Using `temperature`, `top_p`, `top_logprobs` with reasoning — rejected on GPT-6
- Using `reasoning.effort: "none"` on Astra / 6.1 Sol — only Luna accepts it
- Parallel tool calling with custom/freeform tools — not supported
- Splitting free-text across multiple CFG terminals — use one bounded terminal
- Contradictory instructions in the same system prompt
- Complex CFG grammars without testing at high reasoning effort first
- Re-generating full CoT on every turn — use `conversation=` or `previous_response_id`
- Using Chat Completions for tool calling with reasoning on Astra / 6.1 Sol — use the Responses API

📚 Key Resources

Changelog: **developers.openai.com/api/docs/changelog** | Migration & Prompting Guide: **developers.openai.com/api/docs/guides/latest-model** | Model Line: **developers.openai.com/api/docs/models/gpt-6-astra** | Lark IDE: **www.lark-parser.org/ide/**

© 2026 Automata Learning Lab | O'Reilly Live Training: Getting Started with GPT-6 Astra | Professional educational materials crafted with care